

DOCUMENTACIÓN TÉCNICA

# Compara Tech GT

Guía de instalación, arquitectura, scrapers e integración de precios

MVP funcional con API PHP, MySQL, frontend vanilla y base de scrapers  
Python

Versión documentada: 1.0

Repositorio local: /Applications/XAMPP/xamppfiles/htdocs/PricesComparator

# 1. Resumen ejecutivo

Compara Tech GT es un MVP para consultar productos tecnológicos y comparar precios de tiendas de Guatemala. La interfaz consume una API PHP que lee MySQL. Los scrapers Python consultan cada tienda, normalizan resultados y la ingesta actualiza las relaciones tienda-producto, precios e historial.

**Estado actual**

La API, la base demo, la interfaz y el scraper de Kemik están operativos. Intelaf y Pacifiko siguen siendo plantillas. El panel visible de administración es informativo; todavía no es un módulo autenticado.

## Componentes terminados

- Frontend responsive con búsqueda, categorías, ordenamiento, detalle, ofertas e historial con Chart.js.
- API PHP con productos, detalle, precios, historial, categorías y tiendas.
- Esquema MySQL con catálogo, tiendas, precios, auditoría de scrapers y coincidencias pendientes.
- Scraper Kemik con extracción HTML y parser de precios en GTQ.
- Ingesta Python idempotente para actualizar producto\_tienda, precio y scraper\_log.
- Flujo CLI para listar, aprobar o rechazar coincidencias pendientes.

# 2. Arquitectura y flujo de datos

El navegador no consulta directamente MySQL. La aplicación web usa rutas relativas hacia api/, y Apache reescribe esas rutas hacia backend/api.php. La ingesta Python se conecta a MySQL de forma independiente.

| Capa          | Ubicación                                            | Responsabilidad                                   |
|---------------|------------------------------------------------------|---------------------------------------------------|
| Frontend      | index.html, frontend/js/app.js, frontend/css/app.css | Búsqueda, tarjetas, detalle, ofertas e historial. |
| API           | api/.htaccess, backend/api.php                       | Enrutamiento HTTP, consultas y respuestas JSON.   |
| Configuración | backend/config/config.php, .env                      | Carga de credenciales y conexión PDO.             |
| Scrapers      | scrapers/stores/*.py                                 | Extracción y normalización por tienda.            |
| Ingesta       | scrapers/ingest.py                                   | Emparejamiento, upsert, precios y bitácora.       |
| Revisión      | scrapers/matches.py                                  | Aprobación/rechazo de coincidencias.              |
| Persistencia  | database/schema.sql, database/seed.sql               | Estructura y datos demo.                          |

## Flujo principal

```
Usuario -> index.html -> fetch('api/productos?q=...')
      -> api/.htaccess -> backend/api.php -> MySQL
      -> JSON -> tarjetas y detalle en el navegador

Actualización de precios:
Kemik/Intelaf/Pacifiko -> scraper -> ingest.py -> MySQL -> API -> frontend
```

# 3. Requisitos e instalación local

Entorno probado: macOS con XAMPP, Python 3.13 y MySQL/MariaDB accesible desde el archivo .env.

## 3.1 Apache y MySQL

```
sudo /Applications/XAMPP/xamppfiles/xampp startapache  
sudo /Applications/XAMPP/xamppfiles/xampp startmysql
```

### 3.2 Base de datos

Desde phpMyAdmin en <http://localhost/phpmyadmin>, importar database/schema.sql y después database/seed.sql. Para otro entorno, ejecutar también la migración de sql/20260910\_match\_pendiente\_datos.sql.

### 3.3 Variables de entorno

```
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_NAME=compara_tech_gt  
DB_USER=root  
DB_PASSWORD=
```

#### Seguridad

No versionar .env ni credenciales. En producción usar un usuario MySQL con permisos mínimos, no root, y restringir DB\_HOST a la red necesaria.

### 3.4 Entorno Python

```
cd /Applications/XAMPP/xamppfiles/htdocs/PricesComparator/scrapers  
python3 -m venv .env  
source .env/bin/activate  
python -m pip install -r requirements.txt
```

### 3.5 Verificación

```
http://localhost/PricesComparator/  
http://localhost/PricesComparator/api/productos  
http://localhost/PricesComparator/api/productos/1  
http://localhost/PricesComparator/api/productos/1/historial
```

## 4. API y frontend

Todas las respuestas siguen el formato {success, data, message}. El frontend usa rutas relativas, por lo que debe abrirse por Apache y no mediante file://.

| Ruta                         | Método | Uso                                                           |
|------------------------------|--------|---------------------------------------------------------------|
| api/productos?q=...          | GET    | Listado filtrado. También admite sort=price_desc o sort=name. |
| api/productos/{id}           | GET    | Detalle, tiendas, precio más reciente y especificaciones.     |
| api/productos/{id}/precios   | GET    | Histórico de precios por tienda.                              |
| api/productos/{id}/historial | GET    | Serie diaria mínima para la gráfica.                          |
| api/tiendas                  | GET    | Tiendas activas.                                              |
| api/categorias               | GET    | Categorías activas.                                           |

### Archivos clave del frontend

- index.html: estructura y contenido inicial.
- frontend/js/app.js: fetch, renderizado, búsqueda, detalle y Chart.js.
- frontend/css/app.css: diseño visual y responsive.

#### Mejora recomendada

La búsqueda API usa LIKE sobre la frase completa. Para que 'Ryzen 7600' encuentre 'Ryzen 5 7600', separar el texto en tokens y exigir que cada token aparezca, o implementar FULLTEXT con criterios claros.

## 5. Scrapers e ingesta

Cada tienda debe devolver una lista con el mismo contrato. La extracción debe respetar robots.txt, términos de uso, límites de frecuencia y cualquier API pública disponible. No evadir CAPTCHA ni mecanismos anti-bot.

### 5.1 Contrato de resultado

```
{
  "name": "Procesador Ryzen 5 7600 3.8GHz 6 Núcleos AM5",
  "sku": null,
  "price": 2039.00,
  "currency": "GTQ",
  "available": true,
  "url": "https://www.kemik.gt/...",
  "image": "https://static.kemikcdn.com/..."
}
```

### 5.2 Pruebas actuales

```
python main.py kemik "Ryzen 5 7600"
python ingest.py kemik "Ryzen 5 7600"
python matches.py list
```

### 5.3 Aprobación manual

```
python matches.py approve ID_PENDIENTE ID_PRODUCTO
python matches.py reject ID_PENDIENTE
```

| Archivo         | Estado    | Acción pendiente                                    |
|-----------------|-----------|-----------------------------------------------------|
| stores/kemik.py | Operativo | Mantener selectores y agregar pruebas de regresión. |

| Archivo            | Estado    | Acción pendiente                                                            |
|--------------------|-----------|-----------------------------------------------------------------------------|
| stores/intelaf.py  | Plantilla | Implementar search y get_product con contrato común.                        |
| stores/pacifiko.py | Plantilla | Implementar search y get_product con contrato común.                        |
| core/base.py       | Base      | Agregar retries, backoff, User-Agent configurable y validación HTTP.        |
| ingest.py          | Operativo | Agregar modo dry-run, métricas y locking para evitar ejecuciones paralelas. |

## 6. Modelo de datos

El catálogo separa el producto genérico de su oferta en cada tienda. Esto permite conservar historial aunque cambien precios o disponibilidad.

| Tabla                    | Propósito            | Campos importantes                                       |
|--------------------------|----------------------|----------------------------------------------------------|
| producto                 | Catálogo común       | nombre, modelo, marca, categoría, SKU global.            |
| tienda                   | Tiendas configuradas | nombre, URL, logo, estado.                               |
| producto_tienda          | Oferta por tienda    | producto, tienda, SKU, URL, disponibilidad.              |
| precio                   | Historial            | precio, precio_anterior, moneda, disponible, fecha.      |
| scraper_log              | Bitácora             | inicio, fin, encontrados, actualizados, errores, estado. |
| producto_match_pendiente | Revisión             | resultado detectado, tienda, precio, estado, asociación. |

### Reglas de persistencia

- No insertar una nueva fila de precio si precio y disponibilidad no cambiaron respecto al último registro.
- Usar la clave única producto + tienda para evitar duplicar producto\_tienda.
- No asociar automáticamente modelos parecidos: 7600 y 7600X son productos distintos.
- Los resultados no reconocidos se conservan para revisión, no se descartan.
- Cada ejecución debe terminar en scraper\_log con estado completado o error.

#### Corrección aplicada

La migración `sql/20260910_match_pendiente_datos.sql` agrega a las coincidencias pendientes la tienda, URL, imagen, precio y disponibilidad necesarios para aprobarlas.

## 7. Trabajo faltante para terminar el montaje

Priorizar las tareas de esta lista antes de considerar el sistema listo para producción.

| Prioridad | Tarea                        | Criterio de terminado                                                              |
|-----------|------------------------------|------------------------------------------------------------------------------------|
| P0        | Completar Intelaf y Pacifiko | Cada scraper devuelve resultados reales con pruebas y respeta políticas de acceso. |
| P0        | Proteger administración      | Autenticación, autorización por rol, CSRF y bitácora de cada cambio.               |
| P0        | No exponer secretos          | .env fuera de Git, usuario DB mínimo, HTTPS y CORS restringido.                    |
| P0        | Validar ingesta              | Transacciones, retries, timeout, lock distribuido y modo dry-run.                  |
| P1        | Panel de catálogo            | CRUD de productos, marcas, categorías, tiendas y revisión de matches.              |
| P1        | Mejorar búsqueda             | Tokens, filtros, paginación y manejo de caracteres especiales.                     |
| P1        | Observabilidad               | Logs rotativos, alertas cuando una tienda falla y tablero de scraper_log.          |
| P2        | Automatización               | Cron/launchd diario, backup y retención de historial definida.                     |
| P2        | Pruebas                      | Unitarias de parseo, integración API-DB y smoke test del frontend.                 |

## 8. Despliegue y operación

Para una instalación reproducible, separar el servidor web, la base de datos y el proceso de ingesta cuando el volumen aumente. En el MVP pueden convivir en el mismo equipo.

## Checklist de despliegue

- Copiar el repositorio sin .venv y crear un entorno Python nuevo.
- Configurar .env con credenciales de producción y probar conexión PDO y mysql-connector.
- Importar schema, seed solo en entornos de desarrollo y la migración correspondiente.
- Configurar Apache con mod\_rewrite y verificar api/.htaccess.
- Ejecutar una ingesta dry-run antes de escribir datos.
- Programar la ingesta con un usuario del sistema sin privilegios administrativos.
- Guardar logs fuera del webroot y configurar backup de MySQL.
- Verificar que la fecha del último scraper\_log sea reciente y que la API entregue datos.

## Comando operativo recomendado

```
cd /Applications/XAMPP/xamppfiles/htdocs/PricesComparator/scrapers
source .venv/bin/activate
python ingest.py kemik "Ryzen 5 7600"
python matches.py list
```

## 9. Diagnóstico y errores frecuentes

| Síntoma               | Causa probable                                          | Solución                                                                    |
|-----------------------|---------------------------------------------------------|-----------------------------------------------------------------------------|
| data: [] en API       | La frase no coincide completa o no hay seed.            | Probar /api/productos sin q y luego una búsqueda con el nombre completo.    |
| Scraper devuelve []   | URL/parametro o selector HTML desactualizado.           | Verificar status, URL final, robots.txt y HTML real de la tienda.           |
| No conecta MySQL      | .env incorrecto, puerto cerrado o usuario sin permisos. | Probar credenciales y conectividad; revisar logs de MySQL.                  |
| Pendiente sin datos   | Fue creado antes de la migración.                       | Ejecutar nuevamente ingest.py; el código actual rellena el registro legado. |
| API 500               | PDO, esquema o SQL incompatible.                        | Revisar error_log de Apache y confirmar tablas/columnas.                    |
| Frontend no carga API | Se abrió index.html con file:// o Apache no reescribe.  | Abrir http://localhost/PricesComparator/ y verificar mod_rewrite.           |

## 10. Referencia rápida de archivos

| Ruta                                   | Descripción                    |
|----------------------------------------|--------------------------------|
| README.md                              | Requisitos y alcance del MVP.  |
| index.html                             | Página principal.              |
| backend/api.php                        | Controlador de endpoints JSON. |
| backend/config/config.php              | Carga .env y conexión PDO.     |
| database/schema.sql                    | Esquema completo.              |
| database/seed.sql                      | Datos demo.                    |
| scrapers/main.py                       | Prueba directa del scraper.    |
| scrapers/ingest.py                     | Persistencia de resultados.    |
| scrapers/matches.py                    | Revisión de coincidencias.     |
| scrapers/core/product_matcher.py       | Normalización y matching.      |
| sql/20260910_match_pendiente_datos.sql | Migración de pendientes.       |

## Conclusión

El MVP ya puede servir catálogo demo, consultar la API y actualizar ofertas de Kemik. Para terminar el montaje funcional, la prioridad es completar los dos scrapers restantes, agregar el panel seguro de administración y automatizar una ingesta observada y repetible. Esta documentación refleja el estado real del código y separa claramente lo operativo de lo pendiente.